

Caio Collino Troiano
Gabriel Rosa Leite Barroso
Lucas da Mata Guimarães
Thiago Pereira Lins da Silva
Vinicius Vianna Egydio de Carvalho

Trabalho Final - Análise de Algoritmos

São Paulo
2026

Sumário

1	ROTTING ORANGES	2
1.1	Estratégia de Solução	2
2	NEAREST EXIT FROM MAZE	3
2.1	Estratégia de Solução	3
3	ICE STATUES FESTIVAL	5
3.1	Estratégia de Solução	5
4	ROUTE CHANGE	7
4.1	Estratégia de Solução	7
5	BEAUTIFUL ARRANGEMENT	8
5.1	Estratégia de Solução	8
6	DIFFERENT WAYS TO ADD PARENTHESES	9
6.1	Estratégia de Solução	9
7	ASSIGN COOKIES	10
7.1	Estratégia de Solução	10
8	PARTITION LABELS	11
8.1	Estratégia de Solução	11

1 Rotting Oranges

Problema: Rotting Oranges (LeetCode)

Paradigma: Grafos (Busca em Largura / BFS Multi-source)

Implementação: (inserir o link pra qnd tiver o git)

Complexidade: (inserir complexidade)

1.1 Estratégia de Solução

O problema foi resolvido modelando a matriz $m \times n$ como um **grafo não-direcionado**, onde:

- Cada célula da matriz nas coordenadas (i, j) funciona como um vértice;
- As conexões diretas (cima, baixo, esquerda, direita) funcionam como as arestas.

Como a podridão se espalha de forma uniforme e simultânea a partir de várias fontes, a técnica ideal é a Busca em Largura Multi-Source (BFS). O uso de uma fila é essencial nessa abordagem, pois garante o processamento nível a nível, o que é fundamental para descobrirmos o tempo mínimo de infecção considerando que já existem várias laranjas podres no início. O algoritmo funciona em três etapas:

1. **Mapeamento Inicial:** O algoritmo percorre a matriz para contar o número total de laranjas frescas e insere a posição de todas as laranjas podres iniciais na fila da BFS;
2. **Propagação por Minuto:** A fila é processada nível por nível (onde cada nível completo representa 1 minuto). Para cada laranja podre retirada da fila, o algoritmo infeta os seus vizinhos frescos válidos, mudando o estado deles para podre e colocando-os na fila para o minuto seguinte;
3. **Verificação Final:** Quando a fila fica vazia, se o contador de laranjas frescas ainda for maior que zero, significa que algumas laranjas ficaram isoladas no grafo, retornando -1. Caso contrário, retorna o total de minutos decorridos.

2 Nearest Exit from Maze

Problema: [Nearest Exit from Maze \(LeetCode\)](#)

Paradigma: Grafos (Busca em Largura / BFS Multi-source)

Implementação: (inserir o link pra qnd tiver o git)

Complexidade: (inserir complexidade)

2.1 Estratégia de Solução

O problema foi resolvido modelando a matriz $m \times n$ (o labirinto) como um grafo não-direcionado e não-ponderado, onde:

- Cada célula livre (caractere '.') da matriz nas coordenadas (i, j) funciona como um vértice;
- As conexões diretas (cima, baixo, esquerda, direita) entre células livres adjacentes funcionam como as arestas de peso unitário.

Como o objetivo principal é encontrar o caminho mais curto (menor número de passos) de um ponto de origem até à borda do labirinto, a técnica ideal é a Busca em Largura (BFS). O uso de uma fila (queue) é essencial nesta abordagem, pois garante uma expansão em "ondas" (nível a nível). Isto é fundamental para o problema, pois assegura que a primeira célula de saída encontrada seja, garantidamente, a mais próxima. O algoritmo funciona em três etapas principais:

1. **Mapeamento Inicial:** O algoritmo recolhe as coordenadas da entrada (entrance) e insere-as na fila da BFS com uma contagem de zero passos. Para otimizar o espaço espacial (evitando criar uma matriz extra de visitados), a própria célula de entrada é imediatamente marcada no labirinto original como visitada (mudando o caractere para '+'). Note-se que a célula de entrada, mesmo que esteja na borda, não conta como saída válida;
2. **Busca e Expansão (Passo a Passo):** A fila é processada vértice a vértice. Para cada célula retirada, o algoritmo calcula as posições dos seus quatro vizinhos. Se um vizinho for válido (estiver dentro dos limites e for um caminho livre '.'), o algoritmo marca-o como visitado ('+'). Imediatamente a seguir, verifica se essa célula vizinha se encontra numa das bordas da matriz (linha ou coluna igual a 0, ou igual ao limite máximo). Se for uma borda, a saída mais próxima foi encontrada e o algoritmo retorna de imediato o número de passos atuais mais um. Caso contrário, coloca o vizinho na fila para a expansão do próximo nível;

3. **Verificação Final:** Quando a fila de processamento fica completamente vazia, significa que todos os caminhos possíveis foram explorados. Se a execução chegar a este ponto sem ter retornado nenhum valor na etapa anterior, conclui-se que não existe nenhuma rota possível para uma saída, retornando assim -1 .

3 Ice Statues Festival

Problema: Ice Statues Festival (BeeCrowd)

Paradigma: Programação Dinâmica (Bottom-Up / Tabulação)

Implementação: [Link para Implementação](#)

Complexidade: $\mathcal{O}(M \times N)$

3.1 Estratégia de Solução

O problema foi resolvido modelando a busca pela quantidade mínima de blocos de gelo para atingir o comprimento desejado M como uma tabela de estados de otimização (idêntico ao problema clássico Coin Change), onde:

- Cada índice i do vetor (tabela) funciona como um subproblema, representando um comprimento específico de bloco que se deseja formar;
- O valor armazenado em cada índice i representa a quantidade mínima de blocos de gelo necessários para formar aquele comprimento;
- Como o problema exige encontrar uma resposta ótima (menor número de blocos) e as soluções para valores maiores dependem diretamente das soluções para valores menores (caracterizando **subestrutura ótima** e **sobreposição de subproblemas**), a técnica ideal é a Programação Dinâmica;
- O uso de uma tabela (vetor) construída de baixo para cima (bottom-up) é essencial nessa abordagem, pois armazena os resultados parciais e evita o recálculo redundante, garantindo eficiência mesmo para valores de M de até 1.000.000.

O algoritmo funciona em **três etapas**:

- **Inicialização da Tabela:** O algoritmo cria um vetor dp de tamanho $M + 1$. O caso base é definido como $dp[0] = 0$, estabelecendo que são necessários zero blocos para obter um comprimento de tamanho zero. Todos os outros espaços são inicializados com um valor muito grande (infinito simulado);
- **Construção Bottom-Up:** Através de laços aninhados, o algoritmo percorre a tabela de 1 até M . Para cada posição i , ele avalia todos os blocos disponíveis cujos comprimentos sejam menores ou iguais a i . Utilizando a equação de recorrência $dp[i] = \min(dp[i], dp[i - \text{bloco}] + 1)$, o algoritmo garante que o valor salvo seja sempre a melhor escolha;

- **Verificação Final:** Após preencher toda a tabela, a resposta ótima para o caso de teste estará no último índice ($dp[M]$), bastando imprimi-la. Como o bloco de tamanho 1 sempre está presente, há a garantia de que uma solução viável sempre existirá.

4 Route Change

Problema: Route Change (BeeCrowd)

Paradigma: Grafos (Algoritmo de Dijkstra para Caminho Mínimo com Restrições)

Implementação: ./c/routeChange.c

Complexidade: $O(n^2)$

4.1 Estratégia de Solução

O problema foi resolvido modelando o sistema de estradas do país como um **grafo valorado** e **não-direcionado**, onde:

- Cada cidade representa um vértice (identificado de 0 a $N - 1$);
- Cada estrada representa uma aresta com peso equivalente ao custo do pedágio;
- Como o objetivo é encontrar o custo mínimo para atingir a cidade destino ($C - 1$) a partir de uma cidade de quebra (K), considerando regras específicas para a rota de serviço fixa $(0, 1, \dots, C - 1)$, a técnica ideal é o **Algoritmo de Dijkstra**.

A restrição fundamental do problema diz que, sempre que o veículo interceptar qualquer cidade que pertença à rota de serviço, ele **obrigatoriamente deve seguir a rota sequencial de forma linear** até o destino final, não podendo mais sair dela. O algoritmo funciona em **três etapas**:

1. **Modelagem e Adaptação do Grafo:** O grafo é montado por meio de uma matriz de adjacência (ou lista de adjacência). Para simplificar a restrição da rota de serviço, as arestas que partem de qualquer cidade u pertencente à rota ($0 \leq u < C$) são modificadas: ela só possuirá uma única saída válida direcionada para a cidade seguinte da rota ($u + 1$), com o peso correspondente ao pedágio dessa rota linear. Qualquer outra estrada saindo de u é ignorada, pois o veículo não pode desviar se já estiver na rota;
2. **Processamento do Caminho Mínimo (Dijkstra):** Executa-se o algoritmo de Dijkstra partindo do vértice inicial K (a cidade onde o veículo foi consertado). Utiliza-se um vetor de distâncias `dist` inicializado com infinito e uma fila de prioridades (ou busca linear do menor vértice não visitado devido aos limites moderados de $N \leq 250$) para computar o menor custo cumulativo acumulado até cada vértice do grafo;
3. **Verificação Final:** Quando o algoritmo termina de processar os caminhos, o valor mínimo do custo acumulado para alcançar de forma válida o destino estará contido na posição correspondente à última cidade da rota de serviço (`dist[C - 1]`), sendo impresso como resultado para cada caso de teste.

5 Beautiful Arrangement

Problema: Beautiful Arrangement (LeetCode)

Paradigma: Backtracking

Implementação: ./c/beautifulArrangement.c

Complexidade: $O(n!)$

5.1 Estratégia de Solução

O problema foi resolvido utilizando a técnica de *Backtracking*, na qual permite gerar e testar todas as possíveis permutações dos números de 1 até n , descartando imediatamente aquelas que violam as restrições do enunciado.

Uma disposição é considerada válida quando, para cada posição i , o número colocado nessa posição satisfaz pelo menos uma das seguintes condições: $num \bmod i = 0$ e $i \bmod num = 0$.

A estratégia consiste em construir o arranjo de forma incremental. Para cada posição da sequência, são testados apenas os números ainda não utilizados que satisfazem a condição de divisibilidade exigida. Quando um número válido é encontrado, ele é marcado como utilizado e a busca continua recursivamente para a próxima posição. O algoritmo funciona em **três etapas**:

1. **Escolha de um Número Válido:** Para a posição atual pos , percorrem-se todos os números de 1 até n . Um número só pode ser escolhido se ainda não tiver sido utilizado e satisfizer a condição de divisibilidade em relação à posição.
2. **Exploração Recursiva:** Após selecionar um número válido, ele é marcado no vetor usado e a função é chamada recursivamente para preencher a próxima posição da sequência.
3. **Retrocesso (Backtracking):** Quando a exploração de uma escolha termina, o número é desmarcado para que possa ser reutilizado em outras combinações. Caso todas as posições tenham sido preenchidas ($pos > n$), uma disposição válida foi encontrada e o contador global é incrementado.

O vetor usado é responsável por controlar quais números já fazem parte do arranjo atual, garantindo que cada valor seja utilizado apenas uma vez.

Ao final da execução, o contador contém a quantidade total de arranjos que satisfazem as restrições do problema, sendo esse valor retornado como resposta.

Como, no pior caso, o algoritmo explora todas as permutações possíveis dos n números, sua complexidade de tempo é $O(n!)$.

6 Different Ways to Add Parentheses

Problema: [Different Ways to Add Parentheses \(Leet Code\)](#)

Paradigma: Divisão e conquista

Implementação: [./python/parenthesis.py](#)

Complexidade: $O(2^n)$

6.1 Estratégia de Solução

Divisão e conquista é a estratégia utilizada para resolver o problema. Percorremos a expressão, e toda vez que um operador é encontrada, dividimos a expressão em uma subexpressão à esquerda do operador e a subexpressão à direita. Depois, recursivamente, calculamos todos os resultados possíveis dessas partes.

Após a obtenção dos possíveis resultados da esquerda e da direita, é feita a combinação de cada valor da esquerda com cada valor da direita utilizando o operador atual. Assim, realizamos a simulação de todas as possíveis maneiras de inserir parênteses.

Quando a subexpressão é apenas um número e não possui operadores, ocorre o caso base. Nele, ocorre o retorno do próprio número.

Memoização é utilizada para evitar recalcular múltiplas vezes trechos iguais da expressão, realizando o armazenamento de valores já calculados para cada subexpressão.

7 Assign Cookies

Problema: [Assign Cookies \(Leet Code\)](#)

Paradigma: Greedy

Implementação: [./python/cookies.py](#)

Complexidade: $O(n \cdot \log n + m \cdot \log m)$, onde n é o número de crianças e m é o número de cookies.

7.1 Estratégia de Solução

É utilizado uma abordagem gulosa, onde ordenamos o vetor de fatores de ganância das crianças e o vetor de tamanhos dos cookies em uma ordem crescente. Em seguida, usamos dois índices: um para percorrer as crianças (variável `child`) e outro para percorrer os cookies (variável `cookie`).

A principal estratégia é tentar satisfazer primeiro as crianças menos exigentes, utilizando o menor cookie possível. Se o cookie atual possuir tamanho suficiente para satisfazer a criança atual, então contamos essa criança como satisfeita e avançamos para a próxima criança. Caso contrário, pulamos esse cookie e utilizamos o próximo, uma vez que ele não será suficiente para nenhuma criança com fator de ganância maior. Assim, evitamos desperdiçar cookies grandes com crianças que poderiam ser satisfeitas com cookies menores, maximizando o número total de crianças satisfeitas. A complexidade de tempo da solução é dominada pela ordenação dos vetores.

Após a ordenação, os dois vetores são percorridos uma única vez, com custo $O(n + m)$. O espaço extra utilizado é $O(1)$, desconsiderando o espaço interno usado pelo algoritmo de ordenação.

8 Partition Labels

Problema: Partition Labels (Leet Code)

Paradigma: Greedy

Implementação: `./python/partitionLabels.py`

Complexidade: $O(n)$

8.1 Estratégia de Solução

O problema foi resolvido utilizando uma estratégia **gulosa (greedy)**, cujo objetivo é particionar a *string* no maior número possível de segmentos, garantindo que cada caractere apareça em apenas uma partição.

Inicialmente, é realizado um pré-processamento da string para armazenar a **última ocorrência** de cada caractere. Dessa forma, para qualquer letra, é possível determinar até qual posição ela ainda precisa estar presente dentro da partição atual. O algoritmo percorre a string mantendo os índices *i*, que é o início da partição atual, e *end*, no qual trata-se da última posição que a partição atual deve alcançar para conter todas as ocorrências dos caracteres já encontrados.

Durante a varredura da partição, cada novo caractere encontrado pode possuir uma última ocorrência mais distante. Nesse caso, o valor de *end* é atualizado para essa nova posição, expandindo a partição.

Quando todos os caracteres compreendidos entre o início da partição e a posição *end* foram analisados, tem-se a garantia de que nenhuma letra dessa partição aparecerá posteriormente na string. Assim, a partição pode ser encerrada com segurança. O algoritmo funciona em **duas etapas**:

1. **Determinação das Últimas Ocorrências:** Percorre-se a string uma única vez para registrar a última posição de cada caractere em uma estrutura de mapeamento.
2. **Construção Gulosa das Partições:** A partir do início de cada partição, define-se inicialmente seu limite como a última ocorrência do primeiro caractere. Em seguida, todos os caracteres dentro desse intervalo são analisados. Caso algum deles possua uma última ocorrência mais distante, o limite da partição é expandido. Quando o intervalo é completamente percorrido, o tamanho da partição é calculado e armazenado, iniciando-se então uma nova partição.

Ao final do processamento, o vetor de resposta contém os tamanhos de todas as partições válidas encontradas.